AI WEATHERWISE

INTELLIGENT WEATHER FORECASTING AND ANALYSIS SYSTEM USING WEATHER API

A 50-Page College Project Report

Submitted in partial fulfillment of the requirements for the award of the degree

Department of Computer Science / Information Technology

Academic Year 2026–2027

CERTIFICATE

This is to certify that the project entitled “AI WeatherWise – Intelligent Weather Forecasting and Analysis System Using Weather API” is a bonafide project work carried out as part of the academic requirements. The work demonstrates the design of a web-based weather information system that consumes weather API data and applies data-processing and machine-learning techniques for useful weather insights.

Guide Signature: _____ Head of Department: _____

Date: _____ Institution Seal: _____

DECLARATION

I hereby declare that the project report titled “AI WeatherWise” is prepared for academic purposes. The report describes the design, implementation approach, API integration, data processing, prediction module, testing strategy and future enhancement possibilities of the proposed system.

The project is intended as an educational prototype. Weather information is obtained from external data services and should not be treated as a replacement for official emergency or meteorological warnings.

ACKNOWLEDGEMENT

I express my sincere gratitude to the management, principal, head of department and project guide for their guidance and encouragement. I also thank the faculty members and friends who provided suggestions during the planning, development and documentation stages.

I acknowledge the Open-Meteo documentation and related open weather-data resources used for understanding API requests, forecast variables and geocoding.

ABSTRACT

AI WeatherWise is a web-based intelligent weather application designed to present current weather conditions, hourly forecasts, daily forecasts and analytical insights in a simple dashboard. The system obtains geographic coordinates through a geocoding service and retrieves weather variables through a weather forecast API.

The application can process temperature, humidity, precipitation probability, rainfall, wind speed, cloud cover and weather codes. An optional machine-learning layer can classify weather conditions or estimate a derived risk indicator from historical/API data. Charts and summaries make the raw JSON response easier to understand.

The proposed system demonstrates how API integration, data processing, visualization and AI techniques can be combined into a practical student project.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Existing System

5. Proposed System

6. Literature Survey

7. Requirements

8. System Architecture

9. Methodology

10. API Integration

11. AI/ML Module

12. Database Design

13. System Design

14. Implementation

15. Testing

16. Results

17. Applications

18. Limitations

19. Future Enhancement

20. Conclusion

References

LIST OF FIGURES AND TABLES

Figure 1 – Overall System Architecture

Figure 2 – User Request Flow

Figure 3 – API Integration Flow

Figure 4 – ML Processing Pipeline

Figure 5 – Database ER Structure

Figure 6 – Dashboard Layout

Table 1 – Functional Requirements

Table 2 – Non-Functional Requirements

Table 3 – API Variables

Table 4 – Test Cases

CHAPTER 1 – INTRODUCTION

Weather affects transportation, agriculture, education, tourism, outdoor activities and many business decisions. Conventional weather applications often provide large amounts of information, but users may still need a concise interpretation of what the data means.

AI WeatherWise focuses on converting machine-readable weather information into a user-friendly experience. The system accepts a location, resolves it to latitude and longitude, requests forecast data, processes the response and displays meaningful weather indicators.

The project also introduces an AI/ML component so that students can study how historical weather variables may be used for classification or derived prediction tasks.

INTRODUCTION – BACKGROUND

Modern weather applications rely on numerical weather prediction models and data services. An application developer does not normally generate atmospheric forecasts from first principles; instead, the application consumes forecast data from a specialized provider and adds application-specific presentation and analytics.

Open-Meteo documents a forecast endpoint using latitude and longitude and supports numerous hourly variables, including temperature, relative humidity, precipitation, weather code, cloud cover and wind speed. □cite□turn0search1□

This architecture makes the project achievable for students while still demonstrating real-world API consumption and data engineering.

PROBLEM STATEMENT

Users need weather information that is fast to access, easy to interpret and organized around a selected location. Raw API responses contain arrays, codes and numerical variables that are useful to software but less convenient for ordinary users.

The problem addressed by AI WeatherWise is to create an application that retrieves reliable forecast data, validates the response, transforms it into readable information and optionally provides AI-assisted interpretation.

The system must also handle invalid locations, API failures, missing values and network errors without crashing.

OBJECTIVES

- Build a responsive weather dashboard.
- Search a location and obtain geographic coordinates.
- Retrieve current, hourly and daily forecast information.
- Display temperature, humidity, precipitation and wind indicators.
- Visualize forecast trends using charts.
- Implement an AI/ML analysis layer using suitable historical or generated training data.
- Provide clear error handling and input validation.
- Design a modular architecture that can be extended later.

SCOPE OF THE PROJECT

The project scope includes location search, weather API integration, forecast display, chart visualization, data storage for selected historical records and an optional ML prediction/classification module.

The scope does not include operating a meteorological model, issuing official disaster warnings, controlling physical devices or guaranteeing predictions beyond the quality of the upstream weather data.

The system is primarily intended as an academic prototype and can be adapted for web or mobile deployment.

EXISTING SYSTEM

Existing weather applications include websites, mobile apps and smart assistants. They commonly show temperature, precipitation, wind, humidity and multi-day forecasts.

However, many basic student applications simply display an API response without data validation, modular architecture or an analytical layer. They may also lack a clear separation between location search, weather retrieval, visualization and prediction.

AI WeatherWise addresses these software-design gaps by separating the system into modules and defining a repeatable data pipeline.

PROPOSED SYSTEM

The proposed system contains five major layers: presentation, application logic, API service integration, data/ML processing and storage.

A user enters a city name. The application calls the geocoding service to find matching locations. The selected coordinates are sent to the forecast API. The JSON response is validated and transformed into dashboard cards, tables and charts. Historical records can be stored for analysis.

An ML component may use features such as temperature, humidity, precipitation, cloud cover and wind speed to classify a weather category or generate a derived advisory score.

LITERATURE SURVEY

Weather applications commonly use REST APIs because HTTP requests and JSON responses are simple to integrate across programming languages. Geocoding services translate human-readable location names into geographic coordinates.

Open-Meteo's geocoding documentation describes a search endpoint that accepts a location name or postal code and returns matching locations in JSON. □cite□turn0search0□

The forecast API documentation lists current, hourly and daily variables, allowing developers to select only the measurements required by the application. □cite□turn0search1□

LITERATURE SURVEY – DATA SOURCES

Historical weather services can support model development and retrospective analysis. Open-Meteo documentation describes historical weather data and date-based queries using latitude, longitude, start date and end date. □cite□turn0search3□

For an academic prototype, the developer should keep a clear distinction between upstream forecast data and locally generated ML predictions. Model accuracy should be evaluated on a held-out dataset rather than assumed from the API forecast itself.

REQUIREMENTS – HARDWARE

Minimum hardware: dual-core processor, 4 GB RAM, 10 GB free storage and an internet connection.

Recommended hardware: modern quad-core processor, 8 GB or more RAM and stable broadband/mobile internet.

Client devices may include laptops, desktops, tablets or smartphones with a modern browser. Server requirements depend on deployment scale because the prototype performs relatively lightweight API processing.

REQUIREMENTS – SOFTWARE

Operating System: Windows, Linux or macOS.

Frontend: HTML5, CSS3, JavaScript or React.

Backend: Python with Flask/FastAPI.

Data processing: Pandas and NumPy.

Machine learning: Scikit-learn.

Database: SQLite or MySQL.

Visualization: Chart.js or Matplotlib.

Editor: Visual Studio Code.

API testing: Postman or browser developer tools.

FUNCTIONAL REQUIREMENTS

FR1 – The system shall accept a location search query.

FR2 – The system shall display matching locations.

FR3 – The system shall request forecast data for selected coordinates.

FR4 – The system shall show current conditions.

FR5 – The system shall show hourly/daily forecast information.

FR6 – The system shall display charts.

FR7 – The system shall handle API errors.

FR8 – The system shall support AI/ML analysis where training data is available.

NON-FUNCTIONAL REQUIREMENTS

Performance: common requests should be processed quickly under normal network conditions.

Usability: information should be readable on mobile and desktop screens.

Reliability: API failures should produce understandable messages.

Maintainability: modules should have clear responsibilities.

Security: user inputs should be validated and secrets should not be hard-coded.

Scalability: the backend should support additional locations and features later.

SYSTEM ARCHITECTURE

The architecture follows a client–server pattern. The browser communicates with the application backend. The backend performs location lookup and weather-data retrieval, validates responses and sends normalized data to the frontend.

Architecture flow:

User → Web UI → Backend/API Controller → Geocoding API → Coordinates → Weather Forecast API → JSON Parser → Database/ML Module → Dashboard.

This modular structure makes it easier to replace the frontend, weather provider or ML model without redesigning the entire system.

DATA FLOW DIAGRAM

Level-0 DFD:

User → AI WeatherWise System → Weather API → Weather Result → User.

Level-1 DFD:

1. Location Input → 2. Geocoding Process → 3. Coordinate Selection → 4. Forecast Request → 5. JSON Validation → 6. Data Processing → 7. Visualization → 8. AI/ML Analysis → 9. Dashboard.

Each process should log only necessary diagnostic information and avoid storing sensitive user data.

API INTEGRATION

The weather integration uses HTTP GET requests. A forecast request includes latitude and longitude plus selected weather variables. The documented API supports current, hourly and daily weather information. □cite□turn0search1□

Example conceptual request:

GET /v1/forecast?latitude=&longitude;=&current;=temperature_2m,relative_humidity_2m, wind_speed_10m

The response is JSON. The backend extracts the fields needed by the interface and converts them into application objects.

GEOCODING API

A city name cannot be sent directly as latitude and longitude to the forecast endpoint, so the application first resolves the name using a geocoding service.

The Open-Meteo geocoding endpoint accepts a name and returns matching locations. It supports result counts, language and country-code filtering. □cite□turn0search0□

Example conceptual request:

GET /v1/search?name=Coimbatore&count;=5&language;=en&format;=json

The user selects the correct result before the forecast request is made.

WEATHER VARIABLES

Important variables include temperature_2m, relative_humidity_2m, apparent_temperature, precipitation_probability, precipitation, weather_code, cloud_cover, pressure_msl, wind_speed_10m and wind_gusts_10m.

The API documentation also exposes daily aggregations such as minimum/maximum temperature, sunrise, sunset, precipitation sum and maximum precipitation probability. □cite□turn0search1□

The project should request only the variables required by the dashboard to keep the data pipeline simple.

API RESPONSE PROCESSING

The JSON response normally contains metadata such as latitude, longitude, elevation and timezone, together with current/hourly/daily objects.

Processing steps: parse JSON → verify HTTP status → verify required keys → handle null values → normalize units → convert arrays to rows → store selected fields → send clean data to UI.

Validation is important because network failures, changed responses or missing fields can otherwise cause runtime errors.

AI/ML MODULE

The AI component is designed as an analytical layer rather than a replacement for professional meteorological forecasting. A supervised model can classify weather states from historical observations.

Example target classes: Clear, Cloudy, Rainy and Storm-like conditions. Example features: temperature, humidity, cloud cover, precipitation and wind speed.

Suitable beginner models include Decision Tree, Random Forest or Logistic Regression. The selected model should be evaluated with a test set and appropriate metrics.

DATA PREPROCESSING

Data preprocessing includes removing duplicate rows, handling missing values, checking units, encoding categorical labels and scaling features when required by the chosen model.

Example pipeline: raw dataset → cleaning → feature selection → train/test split → model training → validation → evaluation → model serialization.

Data leakage must be avoided. Test data should not influence model training or feature engineering decisions that depend on the target.

MODEL TRAINING

A Random Forest classifier can be used as an example because it can model nonlinear relationships and is relatively accessible for a student project.

Training procedure: load dataset, select features, encode target, split data, fit classifier, calculate evaluation metrics, save model using a safe serialization method, then load it in the backend.

The exact accuracy should be reported from the project's actual test run rather than invented in the report.

MODEL EVALUATION

Common classification metrics are accuracy, precision, recall and F1-score. A confusion matrix shows how predictions are distributed among classes.

Accuracy = correct predictions / total predictions. Precision measures the proportion of predicted positives that are correct. Recall measures how many actual positives were detected. F1-score combines precision and recall.

For weather classes, class imbalance should be checked because a model can appear accurate while performing poorly on rare conditions.

DATABASE DESIGN

A lightweight database can store selected weather records and model results. Suggested tables are Location, WeatherRecord, Prediction and UserPreference.

Location fields: location_id, city_name, latitude, longitude, country.

WeatherRecord fields: record_id, location_id, timestamp, temperature, humidity, precipitation, wind_speed, weather_code.

Prediction fields: prediction_id, record_id, predicted_class, confidence, model_version.

Foreign keys maintain relationships between locations, observations and predictions.

ER DIAGRAM DESCRIPTION

Entities:

LOCATION 1 —— M WEATHER_RECORD

WEATHER_RECORD 1 —— 0..M PREDICTION

Location identifies the place. WeatherRecord stores normalized weather observations. Prediction stores AI/ML output linked to an observation or feature window.

This design avoids storing repeated location details in every weather record and supports historical analysis.

USER INTERFACE DESIGN

The dashboard should use a simple layout: search bar at the top, selected location information, current weather card, forecast cards, hourly chart, precipitation indicator and AI insight section.

Responsive design allows the same interface to work on mobile and desktop screens. Icons and labels should be used together so that the interface remains understandable without relying only on color.

Accessibility considerations include readable font sizes, keyboard-friendly controls and meaningful alternative text for visual elements.

IMPLEMENTATION – FRONTEND

HTML/CSS/JavaScript can implement the prototype without a heavy framework. A React version can separate the interface into SearchBar, CurrentWeather, ForecastCard, WeatherChart and AIInsight components.

The frontend should call the backend rather than exposing sensitive server credentials. API responses should be rendered only after validation.

Chart.js can display temperature and precipitation trends from normalized arrays.

IMPLEMENTATION – BACKEND

A Flask backend can expose routes such as /search, /weather and /predict. The /search route calls geocoding, /weather retrieves forecast information, and /predict loads the ML model and returns the prediction.

Backend responsibilities include request validation, external API communication, timeout handling, response parsing, logging and consistent JSON responses.

Example response structure: { location, current, hourly, daily, ai_insight }.

SECURITY CONSIDERATIONS

Validate and sanitize location input. Use HTTPS in production. Apply reasonable request limits. Do not expose private API credentials in frontend source code. Avoid logging unnecessary personal information.

If a commercial weather service is used, its API key should be stored as an environment variable. The project should also respect the provider's license, attribution and usage terms.

Database queries should use parameterized statements to reduce injection risk.

ERROR HANDLING

Possible errors include empty location input, no matching city, network timeout, API rate limit, malformed JSON and unavailable ML model.

User-facing messages should be simple: “Location not found”, “Weather service temporarily unavailable” or “Prediction is currently unavailable.” Detailed technical errors can be recorded in server logs.

Fallback behavior may show cached data when appropriate, while clearly identifying that the information is not live.

TESTING STRATEGY

Testing covers unit, integration, system, UI and error-handling scenarios. API integration tests verify that valid coordinates return expected fields. UI tests verify that cards and charts render correctly.

ML tests verify that the model accepts the correct feature schema and returns a valid class. Regression tests should be repeated whenever API parsing or model features change.

Testing should be performed using both valid and invalid inputs.

TEST CASES

TC01 – Valid city search → matching locations displayed → PASS/FAIL.

TC02 – Unknown city → not-found message → PASS/FAIL.

TC03 – Valid coordinates → weather data displayed → PASS/FAIL.

TC04 – Network unavailable → graceful error → PASS/FAIL.

TC05 – Missing API field → parser handles safely → PASS/FAIL.

TC06 – ML prediction → valid class returned → PASS/FAIL.

TC07 – Mobile layout → no horizontal overflow → PASS/FAIL.

RESULTS AND DISCUSSION

The expected result is a dashboard that retrieves weather information from the API and presents it in an understandable form. The API documentation confirms that current and hourly variables can include temperature, humidity, precipitation, weather code and wind information. □cite□turn0search1□

Actual project screenshots should be inserted in this section after running the application. Recommended screenshots include location search, current-weather dashboard, hourly chart, daily forecast and AI insight panel.

Model results should be reported from the actual experiment, including dataset size, split ratio, metrics and confusion matrix.

APPLICATIONS

Travel planning, agriculture support, outdoor event planning, educational weather dashboards, logistics awareness, smart-city demonstrations and personal activity planning.

The project can also serve as a learning platform for REST APIs, JSON processing, data visualization, Python backend development and introductory machine learning.

It should not be represented as an official warning system unless integrated with an appropriate authoritative warning source and validated for that purpose.

ADVANTAGES

Simple location-based interface; real-time retrieval from an external weather service; modular backend; visual forecast trends; optional AI analytics; extensible database design; responsive UI; educational value across API, web and ML topics.

The separation between external forecast data and local analytics makes the system easier to maintain and test.

LIMITATIONS

Forecast quality depends on the upstream weather service and underlying numerical weather prediction models. The ML module is limited by the quality, quantity and representativeness of training data.

Internet connectivity is required for live API access. API availability, usage limits and licensing conditions may change. The student prototype is not intended to replace official meteorological services or emergency alerts.

FUTURE ENHANCEMENT

Future versions can add weather alerts, personalized notifications, multilingual support, map-based location selection, PWA/mobile deployment, historical trend dashboards, ensemble model comparison and additional weather providers.

An advanced version could compare multiple forecast sources, quantify uncertainty and use time-series models. A carefully designed recommendation layer could also provide activity suggestions based on weather thresholds.

PROJECT MANAGEMENT

Suggested phases: requirement analysis, UI design, API research, backend development, frontend integration, database design, ML experimentation, testing, documentation and final presentation.

Version control should be used for source code. A simple issue list can track bugs and enhancement requests. Each milestone should produce a demonstrable output so that integration risks are discovered early.

COST AND FEASIBILITY

The project is technically feasible for a student team because the prototype can be built using common open-source development tools and an accessible weather API.

Major costs depend on hosting, commercial API usage, database hosting and optional domain registration. A local prototype can be developed without these deployment costs.

Operational feasibility is high for a demonstration because users interact with a familiar search-and-dashboard workflow.

CONCLUSION

AI WeatherWise demonstrates a practical approach to building an intelligent weather application by combining geocoding, weather API integration, data processing, visualization and machine-learning analysis.

The project provides a complete software-development case study: requirements are translated into modules, API responses are validated, weather variables are transformed into user-friendly information and AI techniques are evaluated separately from upstream forecast generation.

The system can be extended into a larger weather analytics platform while retaining the same modular architecture.

REFERENCES

1. Open-Meteo Forecast API Documentation – https://open-meteo.com/en/docs (accessed September 2026).

2. Open-Meteo Geocoding API Documentation – https://open-meteo.com/en/docs/geocoding-api (accessed September 2026).

3. Open-Meteo Historical Weather API Documentation – https://open-meteo.com/en/docs/historical-weather-api (accessed September 2026).

4. Open-Meteo Forecast API OpenAPI specification – https://github.com/open-meteo/open-meteo/blob/main/openapi/forecast.yml (accessed September 2026).

5. Python Documentation – https://docs.python.org/

6. Scikit-learn Documentation – https://scikit-learn.org/

7. Flask Documentation – https://flask.palletsprojects.com/

APPENDIX – SAMPLE API PARAMETERS

Suggested current variables: temperature_2m, relative_humidity_2m, apparent_temperature, precipitation, rain, weather_code, cloud_cover and wind_speed_10m.

Suggested hourly variables: temperature_2m, precipitation_probability, precipitation, relative_humidity_2m and wind_speed_10m.

Suggested daily variables: weather_code, temperature_2m_max, temperature_2m_min, sunrise, sunset, precipitation_sum and precipitation_probability_max. These variables are documented by Open-Meteo. □cite□turn0search1□

APPENDIX – SAMPLE PROJECT FLOW

1. User enters city.
2. Backend validates text.
3. Geocoding service returns candidate locations.
4. User selects a location.
5. Backend sends latitude/longitude to forecast API.
6. JSON response is validated.
7. Data is normalized and optionally stored.
8. Dashboard displays current/hourly/daily information.
9. ML module generates an analytical result.
10. UI displays the result with an appropriate disclaimer.